# Software Architecture as a Layered Representation

Analyzing Software Architecture with S202

Johannes Weigend

Final version, May 30, 2026

**Abstract**

S202 computes a hierarchical layered representation of Java systems from bytecode. The tool derives an architectural hypothesis from the observed class dependencies, distinguishes two kinds of cycles — class-level cycles and package-level cycles — makes both visible as separately toggled findings, and checks the finished representation redundantly against consistency rules. This document describes typical errors in the computation of such layouts, explains the conceptual solution, and shows how the representation can be used as a working tool for refactoring planning.

## 1 Introduction

At least since the layered models for operating systems in the late 1960s, software architecture has been represented as a layered dependency structure. The principle is still relevant today: in cloud-native architectures it helps to order technical dependencies between services, platforms, and infrastructure; in domain-driven design it helps to make domain boundaries and dependencies between bounded contexts visible. The basic idea is simple: an element that uses another element is placed above the element it uses. Dependencies therefore run from top to bottom. Edges in the opposite direction stand out because they often indicate cycles, unwanted coupling, or a broken layering order.

Such vertically aligned graphs help to understand systems and to detect anomalies in their dependencies. A simple example is provided by the Java tool `jdeps` and the Graphviz tool `dot`: `jdeps` extracts class dependencies from a JAR, and `dot` turns them into a directed graph.

Figure 1 shows an acyclic example from the Example JAR of the S202 codebase. It was created with `jdeps` and `dot`. The JAR contains nine classes in three packages: `example`, `example1`, and `example2`.

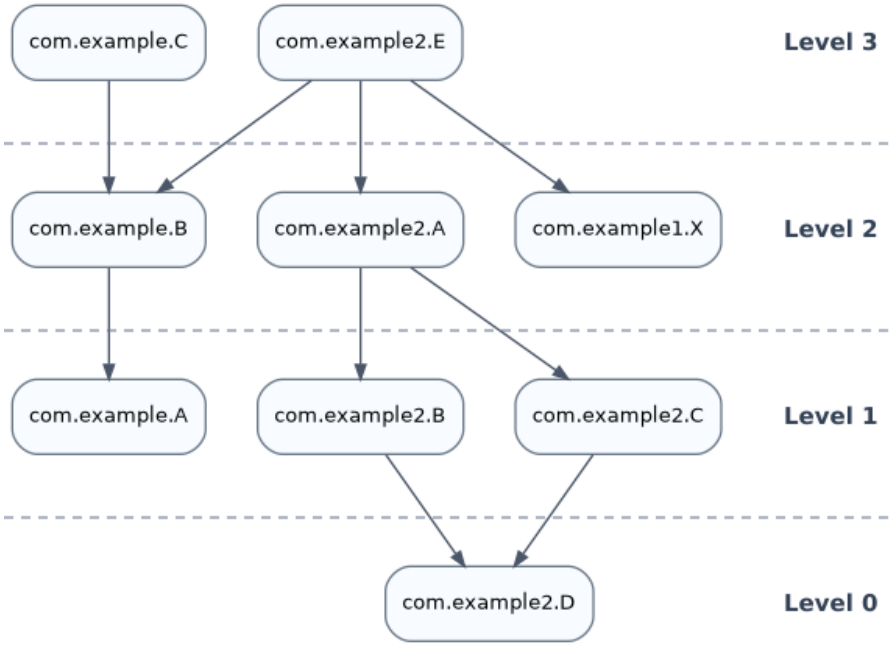


Figure 1: Acyclic example from `test-example-1.0.0.jar`.

The arrows between the boxes are dependencies between classes by invocation or use. In the example, `com.example.B` uses class `com.example.A`, abbreviated as $B \to A$. The horizontal lines were added manually and visualize the levels of the layering. Level 0 forms the foundation; higher levels represent elements that depend on elements below them.

This kind of representation is useful and widely used. As a visualization of real applications, however, it has three practical problems:

1. Many arrows make the graph hard to read even for medium-sized applications.
2. The package structure is spread across several levels and does not form a visible grouping.
3. With cyclic dependencies, the cycle must be broken for level computation so that any vertical order can be created at all. The original dependency graph remains unchanged; only the derived DAG (*Directed Acyclic Graph*) used for level computation is changed. The decisive question is therefore which edge is excluded from this computation, because exactly this edge later appears as a break in the computed order.

## 2 S202 as a Hierarchical Layered Representation

With the open source tool S202, we address these problems directly. S202 does not simply replace the flat class graph with an improved representation. It replaces it with a hierarchical architectural hypothesis: packages remain visible as containers, and inside each container, packages or classes are ordered by their dependencies.

The arrows can be shown optionally. In the normal case, however, the vertical order alone should make it understandable which parts of the application are higher up and which parts serve as the

foundation. Unwanted dependencies therefore do not appear merely as a single line in a large network, but as a break in an expected layering order.

Unlike the flat class graph, S202 structures levels hierarchically through the package structure. Each package orders its direct children, that is, subpackages or classes, according to their local dependencies. The package structure remains visible instead of being pulled apart by the class graph.

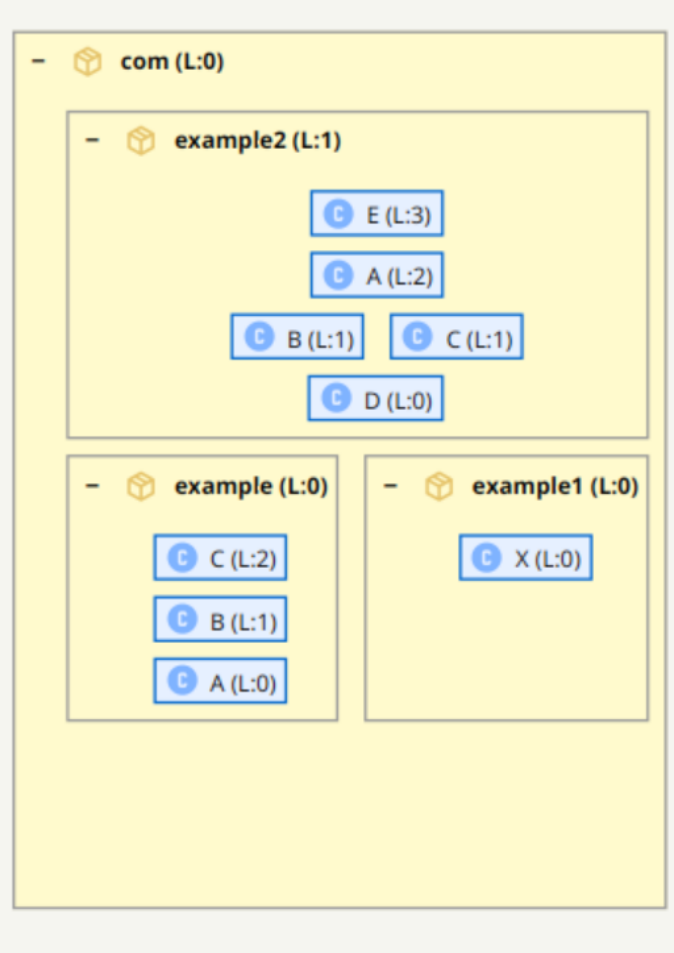


Figure 2: S202 representation of the acyclic example: packages remain visible as containers, classes are ordered within their container, and arrows can be shown when needed.

The example shows the package structure below the outer package `com`: `example`, `example1`, and `example2` are subtrees of this container. In the `example2` subtree there is a non-trivial internal class structure; at the same time, the overarching structure shows that `example2` depends on `example` and `example1`. At class level, `example2` has the order $E \to A \to (B \mid C) \to D$. The representation deliberately abstracts from individual edges. Whether `A` has a dependency to `B`, to `C`, or to both is less important for the coarse architectural view than the layering of the package. This abstraction is exactly what keeps large relationship networks readable.

The idea is not new. The commercial product Structure101 already had a very similar representation. However, there is no open implementation and no technical documentation that explains how such a layout can be computed reliably. S202 closes this gap as an open source tool under the Apache license.

This document describes the typical pitfalls in implementing such a layout and shows a solution that eliminates many of these problems cleanly at the conceptual level.

# 3 Typical Errors in Computing Layers

Figures 1 and 2 deliberately show a simple, acyclic example. In this case the layering is unambiguous: `E` is above `A`, `A` is above `B` and `C`, and both are above `D`. There is no feedback, so no edge has to be selected and ignored for level computation.

In real applications this case is the exception rather than the rule. As soon as classes or packages depend on each other cyclically, a normal graph pass is no longer sufficient. This is where the interesting errors begin: the representation may still look plausible, but the computed levels are no longer stable or no longer explainable in architectural terms.

In the following sections, the term SCC plays a central role. An SCC (*Strongly Connected Component*) is a set of nodes in which every node can reach every other node through directed dependency paths. In such a cycle, not all dependencies can run downward at the same time.

A correct layered layout must therefore satisfy several requirements at once:

1. If $A \to B$, then $A$ must be placed above $B$.
2. Packages must remain visible as containers. A package must not be moved accidentally just because a single contained class has a high class level.
3. Classes or packages that form an SCC must be handled explicitly, because in a cycle not all edges can run downward at the same time.

Each requirement is manageable in isolation. Together, however, they create a circular dependency system between the levels of computation themselves: class levels influence visible positions, package containers structure these positions, and SCCs force corrections to an order that would already have been computed without cycle handling. This is the source of the typical failure modes.

## 3.1 Retroactive Correction

Retroactive correction means here that a structure discovered later forces a subsequent change to levels that have already been computed.

A naive algorithm iterates over the classes and assigns levels directly. It first sees class `A`, assigns its level, and thereby also lifts the package of `A`. Later it finds class `B` and discovers that `A` and `B` belong to the same SCC and must be handled together. Now `A` would have to be corrected, and with it its package and possibly further packages that depend on it.

The problem is not the correction itself. The problem is that parts of the layout have already been treated as finished at that point. An algorithm that mixes cycle detection and level assignment has to change, retroactively, places that it had already left behind.

## 3.2 Order-Dependent Results

Classes come from JARs. The order in which a tool sees classes follows not the architecture, but the packaging order of the JAR, the module order, the build tool, or filesystem details. If an algorithm handles cycles as a side effect during this traversal, the result depends on this accidental order.

This is dangerous because the error does not have to be immediately visible. The graph may still look roughly correct. Nevertheless, the same codebase can receive different levels depending on the build configuration.

### 3.3 Mixed Class and Package Levels

One obvious mistake is to derive the level of a package from the highest level of the classes it contains. This sounds plausible, but it mixes two different questions: where a class lies in the class graph, and where a package lies in the architectural hypothesis.

A package is not architecturally high just because it happens to contain a highly placed class. The central rule is:

> Package levels are derived from package dependencies, not from the maximum height of individual classes.

This sounds trivial, but it is not. Precisely because packages are containers for classes, it is tempting to derive their position from the classes they contain. That is exactly how two levels get mixed that must remain separate. Only when package levels are derived from package dependencies do packages remain stable architectural units. Otherwise a single outlier class is enough to lift the package to the wrong level and thereby make the container unusable, even though that container is supposed to provide orientation.

### 3.4 The Big Lake

The formally clean answer to cycles is: all elements of an SCC receive the same level. For small cycles this is acceptable. In real systems, however, SCCs can become very large. Then hundreds or thousands of classes end up on the same level. The layering disappears; what remains is a flat block.

The cycle is then correctly detected, but no longer usefully visualized. A tool must therefore be able to break large SCCs. The decisive point is that the cut edges must not disappear. They must remain visible, because they explain why the computed order is only an architectural hypothesis.

## 4 Two Kinds of Cycles

S202 distinguishes two fundamentally different kinds of cycles and makes both visible as separately toggled overlays:

**Class cycles (red)** Strongly connected components at the class level. Two or more classes that depend on each other directly or transitively form such a cycle. Class cycles reveal implementation-level coupling that prevents a strict linear order of individual classes. They are detected by Tarjan's algorithm on the class dependency graph.

**Package cycles (orange)** Tangles at the package level, detected on the direct aggregated package graph (without the ancestor propagation described later). A package cycle means that two or more packages depend on each other as a group, regardless of which concrete classes inside them are responsible. Package cycles reveal architectural-level coupling: the package structure itself has circular dependencies.

The distinction matters because the two kinds answer different questions:

- A *class cycle* shows where individual classes need to be untangled. Breaking it often requires moving a method, extracting an interface, or inverting a dependency between two specific classes.
- A *package cycle* shows where package boundaries themselves are broken. Resolving it may require a larger restructuring — merging two packages, introducing a new one, or redefining the module split entirely.

A package cycle can exist even when no individual class cycle is visible — for example, when two packages call each other through different classes that are each individually acyclic. Conversely, a class cycle contained entirely within a single package does not necessarily imply a package cycle.

Because the two overlays answer different questions, S202 keeps them as independent switches. Activating both at the same time is possible and useful when looking for the root cause of an architectural problem, but each overlay is also meaningful on its own: the package-cycle overlay reveals the structural shape of the problem, while the class-cycle overlay reveals the concrete edges that cause it.

## 5 The Package Graph as an Architectural Hypothesis

The solution chosen by S202 is to separate analysis, ordering, and representation consistently. The class graph remains the raw model of the actual dependencies. It is not changed merely because a layered representation requires a DAG. Instead, S202 creates additional views from this graph: SCCs for cycle detection, a weighted package graph for the architectural hypothesis, and a hierarchical UI structure for the representation.

The term architectural hypothesis does not mean an asserted target architecture. It means the actual architecture inferred from the observed dependencies: the order that best fits the existing code and its dependencies without hiding where this order is broken by cycles or back edges.

The package ordering is the central connecting element. It is not derived from the highest class levels, but from aggregated package dependencies. This separation is the decisive step: classes are ordered within their packages, whereas packages are ordered by their own package relationships. This allows a package to remain stable as a container even when individual classes inside it are high or low. At the same time, the package ordering provides a criterion for selecting back edges not arbitrarily, but as concrete violations of the expected order.

Thus, the failure modes described above do not disappear because of one algorithmic trick, but because the computation steps have clear responsibilities: first dependencies are analyzed, then a plausible order is determined, then the visible structure is built, and finally the result is checked to see whether the representation satisfies its own claim.

## 6 From Class Dependencies to Package Ordering

In S202, the flat class graph is not the visible layout model. It is an analysis artifact: it is used to capture concrete dependencies, detect SCCs, aggregate class dependencies into package relationships, and derive the later levels from them. The visible S202 graph is then built from the package structure and the local order inside package containers.

Package ordering is built from a weighted directed graph spanning *all* packages. Classes are not comparison nodes in this step; they provide the information for aggregation. For each class dependency, S202 counts the method calls from the source class to the target class and attributes this count to the corresponding package edge: if a class in the subtree of package $P$ calls a class in the subtree of package $Q$ via $n$ method calls, the edge $P \to Q$ receives weight $n$. If a class has only dependencies without method-call data into a target package, it contributes a fallback weight of 1 to that package edge (per class and target package, not per dependency).

To capture the contribution of nested packages, each weight is propagated up the ancestor chain of the source package: a call from `com.a.X` to `com.b.Y` contributes weight to `com.a`→`com.b` and also to `com`→`com.b`. The transitive order among packages emerges from the subsequent longest-path computation — no additional transitive edges are added to the graph.

Ancestor propagation can, however, create cycles that do not exist in the direct package graph. Package cycles (tangles) are therefore detected on the direct, non-propagated package graph; cycle-breaking and level assignment, by contrast, operate on the propagated graph.

A concrete numerical example: the class dependencies `com.a.A`→`net.b.B` and `com.a.B`→`net.b.B` produce the following edges in the package graph:

```
com.a -(2)-> net.b
com   -(2)-> net.b
```

Only the source side is propagated up the ancestor chain; the target side stays at the direct parent package of the target class. `net.b` is not further propagated to `net`. Propagating the target side as well would produce a cross-product of source and target ancestors for every class dependency: the number of edges grows quadratically with package depth, without any information gain for the ordering computation.

Before level computation, the package graph must be acyclic. If packages form an SCC, S202 uses a rank measure to evaluate whether a package tends more to use other packages or to be used by them:

$$rank(P) = \frac{out(P) - in(P)}{\max(1,\ out(P) + in(P))}$$

$out(P)$ is the sum of the edge weights from $P$ to other members of the same SCC, and $in(P)$ is the sum of the edge weights from those members to $P$. A positive rank means that the package uses more than it is used; it therefore tends to belong higher up. A negative rank means that the package is more often used and tends to belong lower down.

In a package SCC, all edges $P \to Q$ for which $rank(P) < rank(Q)$ holds are treated as back edges – that is, all edges that run against the dependency flow. No threshold is used: any measurable rank difference is sufficient justification for a cut.

If all ranks within an SCC are equal, the topology provides no directional information. In this case, all internal edges are removed and the levels of the affected nodes are determined solely by dependencies outside the former SCC. If no such dependencies exist either, the nodes remain at the same level – as genuine peers with no justifiable ordering.

Both kinds of cuts are applied iteratively: after each pass, the SCCs are recomputed, and for any remaining residual SCCs the ranks are recalculated over their members. The procedure ends when no SCC with at least two members remains.

Symmetry detection is based on the rank measure, not on weight comparison. An SCC in which all edges carry the same weight is not necessarily symmetric: different in- and out-degrees produce different ranks despite equal weights and thereby yield a clear cut direction.

After this cycle handling, S202 computes levels on the resulting DAG. Level 0 contains the packages that do not use any other packages. Every other package is placed one level above the highest package it directly uses. This creates the transitive layering: from $B \to A$ and $C \to B$, it follows that $A$ is on Level 0, $B$ on Level 1, and $C$ on Level 2, even though no direct edge $C \to A$ exists.

The levels produced here are the *architectureLevel* of each package — the architectural hypothesis used in the next step to guide class SCC-breaking. A separate step, run after class levels are assigned, applies the same approach within each parent container independently. It builds a sibling-only weighted graph of the direct children, excluding class back-edges already resolved in the previous step, and derives a *localLevel* for every element. The local level controls visual positioning inside the parent container and has no effect on the architectural hypothesis or violation detection.

# 7 From Class Cycles to Package Ordering

Figure 3 shows a cyclic example from the Example JAR as a classic class graph. The two subgraphs for Notification and Payment each contain feedback.

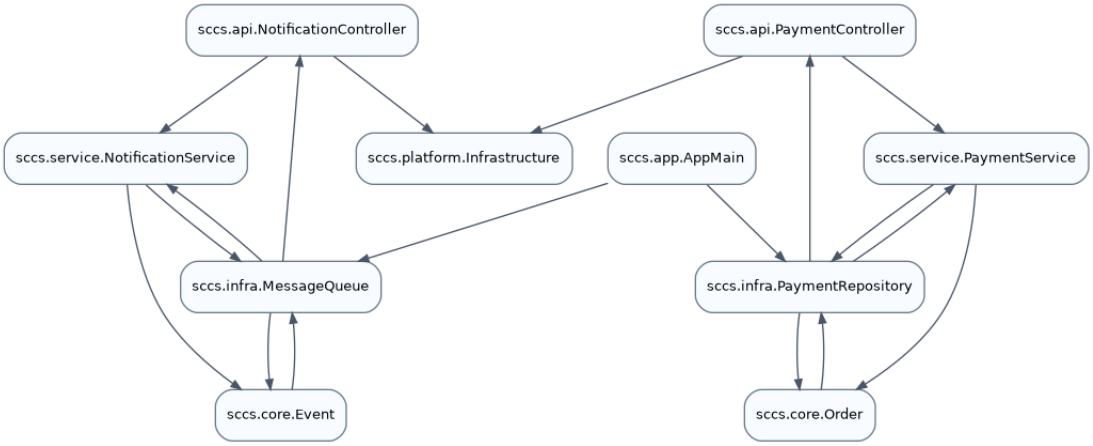


Figure 3: Cyclic example from `test-example-1.0.0.jar`. In every cycle, at least one edge must be cut for level computation.

The marked cyclic areas are two non-trivial SCCs: one in the Notification part of the example, with controller, service, message queue, and event classes, and one in the Payment part, with controller, service, repository, and order classes. The established procedure for finding such components is Tarjan's algorithm.

In the cyclic case, the class graph therefore raises the technical question: which edge must be ignored so that the SCC becomes a computable DAG again for level assignment? The class graph alone does not provide an architectural criterion for that. The package ordering provides this criterion through two cases.

**Case A** covers SCCs whose members span different package levels. An edge $A \to B$ inside such an SCC is cut when $pkgLevel(A) < pkgLevel(B)$ — it runs from a lower-positioned package toward a higher-positioned one, against the architectural direction. The edge is then not an arbitrary cut edge but a concrete dependency that violates the package order. The cut is applied iteratively until no edge inside an SCC runs against the package hypothesis; SCCs that remain afterwards are handled by Case B.

**Case B** covers SCCs confined entirely to a single package level — no package context provides a direction. All internal edges of such an SCC are removed. Class levels for the former cycle members then arise from their dependencies outside the SCC; nodes with no such external dependencies end up at the same level.

All edges removed in both cases are recorded as *classBackEdge* findings and remain visible as dashed violation edges.

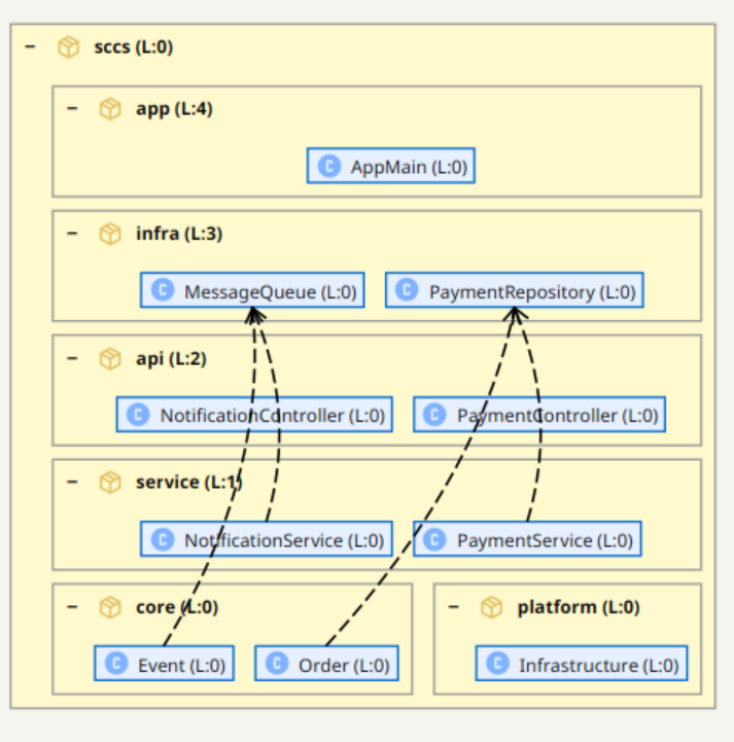


Figure 4: Cyclic example in the S202 representation. The dashed edges are violations: they run against the order derived from package dependencies and remain visible findings of the architectural hypothesis. All four dashed edges belong to the two SCCs and are classified by S202 as back edges for level computation.

The terms back edge and violation describe different roles:

**Back edge** A selected edge inside an SCC that is handled separately for level computation so that the computation graph becomes acyclic. It belongs to cycle handling.

**Violation** A concrete dependency that runs upward after the computed order has been applied. It may come from an SCC, but it does not have to. Violations are therefore a superset of back edges: a back edge can become visible as a violation, but not every violation is a back edge. Conceptually, a violation remains the visible deviation from the ordering, while a back edge describes the role of the edge in cycle handling.

S202 therefore does not hide the cycle. Edges classified as back edges remain visible as violations because they explain why the package ordering does not fully match the concrete class dependencies. This makes the deviation traceable rather than arbitrary.

# 8 Redundant Consistency Checking

The steps described so far already show why the implementation is prone to errors. It is not enough to secure individual algorithms with isolated unit tests. Many errors arise only from the interaction of the data: concrete class dependencies, package structure, SCCs, aggregated package weights, and local layout positions influence one another.

S202 therefore checks more than individual computation steps. After the full pipeline has run, the finished result is checked redundantly against consistency rules. These rules do not recompute the layout and do not correct anything. They only read the result and ask whether the generated representation is internally consistent.

The check considers three levels:

1. **Class graph:** For normal class dependencies, the following must hold: if $A \to B$, then $A$ is on a higher level than $B$. Exceptions must be classified explicitly, for example as a back edge or as an edge inside a class SCC. Class-level cycles detected here correspond to the red overlay in the UI.
2. **Package graph:** The computed package ordering must be consistent with the weighted package dependencies. For every package dependency $P \to Q$ that is not classified as a package back edge, $P$ must lie strictly above $Q$ in the computed ordering. Edges cut during package SCC-breaking must be classified as package back edges and are exempt from this check. Package-level cycles detected here correspond to the orange overlay in the UI.
3. **Visible representation:** The visible arrangement must match the computed model. Package containers, local class positions, and nested levels must not silently reverse the computed architectural hypothesis.

These consistency rules are neither an additional optimization nor a second layout algorithm. They are an independent plausibility check after the calculation has finished. This is exactly why they find errors that normal unit tests easily miss: a package level was computed correctly, but displayed incorrectly locally; a back edge was cut, but not marked as such; a normal edge runs upward without being part of an SCC.

The decisive point is redundancy. The pipeline generates the representation. The rules then check whether this representation satisfies its own claim. This turns a mere graphic into a verifiable model.

# 9 From Visualization to Action

A computed architectural hypothesis is especially useful when it does not only describe a state, but also makes change plannable. S202 contains a what-if analysis for this purpose: classes and packages can be moved in the representation by drag and drop. The model is not refactored immediately; instead, the tool computes which dependencies would run upward in the new arrangement.

The violations are updated after every move. This shows whether a planned reordering makes the architecture more consistent or introduces new problems. This is especially important in situations where the tool can no longer find a unique order: large SCCs, symmetric package dependencies, or historically grown couplings cannot always be resolved automatically in a meaningful way.

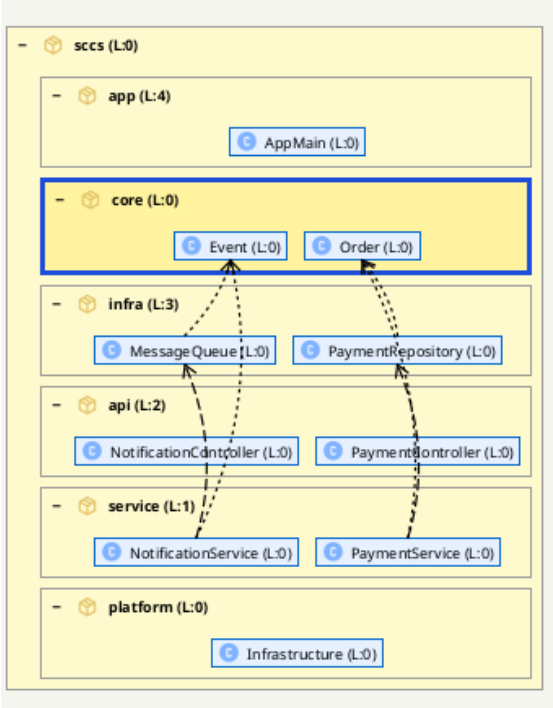


Figure 5: What-if move: package `core` is moved from Level 0 between Level 3 and Level 4. This creates six violations because existing dependencies now run against the chosen arrangement.

Figure 5 shows exactly this workflow: the manual arrangement no longer corresponds to the order computed from the dependencies. It can nevertheless express a desired target architecture, for example if `core` should conceptually lie above the infrastructure. S202 therefore does not treat this decision as an error in the representation, but makes its cost visible: in this case, six concrete relationships would have to be refactored or decoupled so that the dependencies again fit the chosen layering.

In addition, there is a manual CUT logic. Back edges can be marked deliberately as planned cuts, and the tool can then check whether these cuts actually resolve the affected SCC. The user not only sees which edge would be a plausible break in the order, but can also verify the effect of that cut on the cycle.

This CUT logic is more fine-grained than the visible class or package edge. The cut is made at method-call level: not necessarily the entire relationship between two classes disappears, but the concrete method call that causes the cyclic dependency. This makes the feature fit practical refactoring work more closely, because a cycle often arises from a few concrete calls, not from the entire domain relationship between two classes.

## 10 Implementation at the Conceptual Level

The implementation deliberately separates the calculation into several phases. The decisive point is that no phase fixes a visible position before the information that justifies this position is available.

1. **Build the class graph:** A directed class graph is created from the bytecode. This graph remains the domain raw model of the analysis. Only edges that carry at least one relationship of kind `extends`, `implements`, `imports`, `instantiates`, or `calls` enter the ordering

computation; pure type references (field types, parameter and return types, casts) remain in the raw model but are not considered.

2. **Detect SCCs:** Strongly connected components are determined on the class graph using Tarjan. The result is not yet a layout; it is the information which classes can be ordered acyclically and which belong to cycles.

3. **Derive the package graph:** Class dependencies are aggregated into a weighted global package graph. Each method call from a class in package $P$ to a class in package $Q$ contributes to the weight of edge $P \to Q$, propagated up the ancestor chain of $P$. The result is a single flat graph spanning all packages.

4. **Compute package ordering:** The rank measure $rank(P)$ is computed from the weighted incoming and outgoing edges. All edges for which $rank(P) < rank(Q)$ holds are classified as back edges and cut; residual SCCs are processed again with recomputed ranks until the package graph is acyclic. If all ranks within an SCC are equal, all internal edges are removed; levels then arise from dependencies outside the former SCC.

5. **Determine class back edges:** Class SCCs are broken using the package hypothesis. *Case A* — an edge $A \to B$ inside an SCC is cut when $pkgLevel(A) < pkgLevel(B)$: it runs against the architectural direction. *Case B* — when all SCC members share the same package level, no package context provides a direction; all internal edges are removed. In both cases, *cut* means the edge is excluded from the DAG used for level computation but preserved as a *classBackEdge* finding visible as a dashed violation edge.

6. **Assign local display positions:** After class levels are assigned, the *localLevel* of every element is computed. Inside each parent container a sibling-only weighted graph of the direct children is built, excluding class back-edges from Step 5; the sibling graph is acyclic by construction; longest-path yields a *localLevel* per child. *localLevel* governs visual positioning inside the parent box; *architectureLevel* is the global semantic level that drives violation detection.

7. **Check consistency:** Finally, the finished representation is checked redundantly against the consistency rules. Normal dependencies, SCCs, back edges, and violations are distinguished and classified.

In short:

```
Bytecode -> class graph -> class SCCs (red overlay)
         -> package graph -> package SCCs / tangles (orange overlay)
         -> package architectureLevel
         -> class architectureLevel -> localLevel -> consistency
            rules
```

# 11 Conclusion

S202 computes the most plausible hypothesis of the actual architecture from the existing dependencies. The result is a verifiable hierarchical layered representation: class dependencies provide the graph, package dependencies provide a robust order, back edges make necessary breaks visible, and consistency rules check whether the finished representation satisfies its own claim.

A key design decision is the explicit separation of two kinds of cycles. Class cycles and package cycles are detected independently, stored as distinct findings, and exposed through separate toggle switches in the UI. Class cycles (red) show which concrete classes form cyclic dependency

chains; package cycles (orange) show which packages are mutually dependent at the architectural level. Because both views are available simultaneously, the tool can answer two different questions at once: *where* in the package structure is the circular coupling, and *which concrete class dependencies* cause it.

The most important contribution is therefore not an optically improved graph, but the connection of representation, explanation, and changeability. The layering shows a possible order. The marked findings explain where this order is broken by concrete dependencies. What-if moves and manual CUTs make visible which refactorings can reduce these breaks or actually dissolve cycles.

This makes the visualization more than a layout. It is an explainable working model: every position and every highlighted edge must be traceable to a comprehensible decision in the computation, and every planned change can be checked against the same order.